

Plot Panel Execution Flow

Snapshot: 2026-05-18

This note summarizes how the plot panel is produced end to end in Agentic-Sys_v2: specialist tool calls, backend chart/table generation, SSE emission, and frontend rendering expectations.

Summary

The plot panel is driven by structured chart/table events, not by inline markdown in the final chat answer.

High-level flow:

1. A specialist produces numeric findings from data tools.
2. A chart/table enters the pipeline either through an explicit `make_chart(...)` tool call or through the async distiller.
3. The backend stores the chart/table as a KnowledgePoint in `sess.specialist_kb`.
4. At end of turn, `server.py` collects current-turn chart/table KPs.
5. The backend emits typed SSE `chart` events.
6. The frontend renders those events in the plot/reasoning panel.

1. Numeric Findings

Specialists usually get chartable numbers from data tools such as:

- `summarize_trend`
- `summarize_by_group`
- `aggregate_column`
- `batch_aggregate`

These tools return structured or semi-structured numeric outputs. The specialist can either cite those numbers in its answer or pass them into `make_chart(...)` when an explicit chart/table is needed.

2. Explicit Chart Tool Path

The explicit charting tool is defined in:

`tools/data_viz_tools.py`

Factory:

`build_make_chart_tool(specialist_name)`

Each specialist gets a bound `make_chart` tool. The binding captures the specialist name, so the tool knows which specialist KB list to append to.

The tool call shape is:

```

make_chart(
    topic: str,
    kind: str,
    claim: str,
    points: list[dict],
    x_field: str,
    y_fields: list[str],
    source_call: str,
)

```

Supported kind values:

- trend
- bar
- share
- trend_dual
- trend_grid
- table

Plot kinds require at least 4 data points. For 1-3 rows, the intended path is `kind="table"`, which skips PNG rendering and lets the frontend render an HTML table/card.

3. chart_pending Event

When `make_chart(...)` starts, it can emit an immediate SSE event:

`chart_pending`

Payload shape:

```

{
    "specialist": specialist_name,
    "topic": topic,
    "kind": kind,
    "turn_id": turn_id,
}

```

Purpose:

- tells the frontend that a plot is being prepared
- lets the plot panel show a pending/working placeholder
- applies only to explicit `make_chart(...)` calls

Auto-distiller charts do not currently emit `chart_pending`.

4. KnowledgePoint Persistence

Both explicit charts and auto-distilled charts are stored as KnowledgePoint-shaped dicts in:

```
sess.specialist_kb
```

Typical KP chart fields:

```
{
  "topic": "...",
  "claim": "...",
  "numbers": [...],
  "viz": {
    "kind": "...",
    "x_field": "...",
    "y_fields": [...]
  },
  "source_call": "...",
  "captured_at_turn": turn_id,
  "confidence": "high" | "medium" | "low",
  "image_path": "...",      # when PNG exists
  "vega_spec": {...},      # when Vega-Lite spec exists
}
```

For kind="table":

- no PNG is rendered
- `image_path` is absent
- `vega_spec` is absent
- `numbers`, `x_field`, and `y_fields` are preserved for frontend table rendering

5. Backend PNG and Vega Generation

Static chart rendering is implemented in:

```
tools/viz_renderer.py
```

The renderer:

- uses matplotlib with the headless Agg backend
- writes PNG files under:

```
reports/<case_id>/charts/<turn_id>-<topic>.png
```

- creates a Vega-Lite v5 spec through:

```
kp_to_vega_spec(kp)
```

- stores the spec on the KP as:

```
kp["vega_spec"]
```

PNG rendering failures are logged and return `None`. The KP can still be saved without a chart image.

6. Auto-Distiller Chart Path

After a specialist returns, `redacting_tool` schedules an async distiller task:

```
agent_factories/redacting_tool.py
```

The distiller extracts KnowledgePoints from the specialist output. If a KP has both:

```
viz: dict
numbers: list
```

then the backend:

1. builds a Vega-Lite spec
2. renders a PNG when applicable
3. stores `vega_spec`
4. stores `image_path` if rendering succeeds
5. appends the KP to `sess.specialist_kb`

This is why plots can appear even when the specialist did not explicitly call `make_chart(...)`.

7. Server Chart Collection

At end of turn, `server.py` waits for pending distiller tasks before collecting plots.

Collection function:

```
_collect_turn_charts(sess.specialist_kb, turn_id, sess.case_id)
```

Selection rules:

- KP must have `captured_at_turn == current turn_id`
- KP must either:
 - have `image_path`, or
 - have `viz.kind == "table"`

Deduplication:

- key is `(specialist, topic)`
- latest KP wins
- this handles cases where both `make_chart` and the auto-distiller produce the same topic

8. Final chart SSE Event

For every collected chart/table, server emits:

`chart`

General payload:

```
{
  "turn_id": turn_id,
  "specialist": "...",
  "topic": "...",
  "url": "...",
  "claim": "...",
  "source_call": "...",
  "kind": "...",
  "vega_spec": {...} | None,
}
```

For table KPs, payload also includes:

```
{
  "numbers": [...],
  "x_field": "...",
  "y_fields": [...],
}
```

Chart events are emitted before the final answer events:

```
chart
final
agent_message
turn_done
```

9. Serving PNG Files

PNG URLs look like:

```
/api/cases/<case_id>/charts/<filename>
```

Flask route:

```
GET /api/cases/<case_id>/charts/<path:filename>
```

The route serves from:

```
reports/<case_id>/charts/
```

It rejects path traversal attempts such as:

- ..
- leading /
- backslashes

10. Frontend Rendering Contract

The frontend plot panel should listen for:

- chart_pending
- chart

For `chart_pending`:

- show a placeholder keyed by `(turn_id, specialist, topic)`
- replace it when the matching `chart` event arrives

For `chart`:

- if `kind == "table"`, render `numbers` as an HTML table/card
- if `vega_spec` exists, render interactive Vega-Lite if supported
- if `url` exists, render the PNG image from the backend route
- if both `vega_spec` and `url` exist, use Vega-Lite as the interactive path and PNG as fallback

The chat answer itself should remain text-only. Plot rendering belongs to the reasoning/plot panel.

11. Latency Considerations

The plot panel can add latency in several places:

- Explicit `make_chart(...)` is an extra specialist LLM tool round-trip.
- Matplotlib PNG rendering is backend CPU work.
- The server waits for pending distiller tasks before collecting auto-generated charts.
- Many chartable KPs can increase distiller and render time.
- `chart_pending` only covers explicit `make_chart(...)`; auto-distiller charts appear only after distillation finishes.

Recent specialist prompt guidance keeps `make_chart(...)` off the default critical path. Specialists should rely on the auto-distiller for ordinary chartable findings and call `make_chart(...)` only when the user explicitly asks for a chart/table or when the specialist has merged data in a way the distiller cannot reconstruct.